Graduation project

Through-wall presence detection with Wi-Fi CSI

ML/DL data pipeline, models and evaluation strategy

Team: Habiba Ashraf, Youssef Rostom

The four-stage pipeline

1

Data cleaning

Hampel filter and phase sanitization

2

Preprocessing

4 s windows, spectrogram, min-max scaling

3

Model training

SVM, Random Forest, CNN, CNN-LSTM

4

Evaluation

Session-based metrics and latency

Stage 1a: Hampel filter (amplitude cleaning)

How it works

A sliding window moves across each subcarrier amplitude series independently.

It computes the median and the MAD of the values inside the window.

A point further than 3 MADs from the median is flagged as an impulse spike.

Flagged points are replaced with the window median.

What MAD is

MAD = median(|Xi − median(X)|)

A robust measure of how far data points spread from their median.

Unlike standard deviation, a few severe spikes do not distort the threshold.

Implementation: window size 5, MAD scaled by k = 1.4826.

Why it is needed

Raw ESP32 CSI has impulse spikes from transmission retries, power adjustments and antenna switching.

Uncleaned, these artifacts dominate the signal and ruin predictions.

The filter removes burst spikes and keeps the continuous human motion trend.

Stage 1b: phase sanitization

1. Phase unwrapping

Corrects the 2π jumps in raw phase values.

Across the 52 subcarriers, if the jump between neighbours exceeds π, add or subtract 2π.

Python: np.unwrap(raw_phase_array)

2. Linear phase transformation

Fit $y = a \cdot x + b$ over the unwrapped phase and subcarrier indices 1 to 52.

Sanitized phase = unwrapped − $(a \cdot x + b)$.

Removes the slope and offset from packet detection delay and clock misalignment (CFO, SCO).

Python: np.polyfit(idx, phase, 1)

3. Conjugate multiplication

$H_{sanitized} = H1 \cdot \text{conj}(H2)$, using two receiver antennas.

Both antennas share the same clock, so the shared phase noise cancels.

Only the relative phase between antennas remains.

Stage 2a: windows, 400 × 52 matrix and STFT

Sliding window

Continuous stream cut into fixed 4-second chunks.

At 100 Hz, one window holds 400 packets.

50% overlap: step of 2 s, 200 packets.

Overlap augments data and captures motions across window edges.

Window label: majority vote of its packets.

Spectrogram matrix

Each sample is 400 time frames × 52 active OFDM subcarriers.

Every pixel is a subcarrier amplitude.

The CNN reads it as a single-channel image.

STFT (Doppler view)

Splits the signal into short overlapping segments and runs an FFT on each.

Output: a spectrogram with time, frequency and energy.

Reveals Doppler shifts that separate fast motion (a fall) from slow motion (walking).

Stage 2b: min-max normalization

The formula

$X_{norm} = (X - X_{min}) / (X_{max} - X_{min})$

Applied to every 400 × 52 matrix, per window or across the dataset.

Values land in [0, 1]. Implemented with sklearn MinMaxScaler.

Why it matters

Absolute Wi-Fi power changes with how close a person stands to the receiver.

Scaling stops that variance from overpowering the fine movement patterns used for classification.

Stage 2 output

Normalized 400 × 52 samples, one label each.

Ready for classical ML features or for the deep learning models.

Stage 3a: classical ML baseline (SVM / Random Forest)

Statistical features

Computed over time for each subcarrier, so 400 steps become one summary.

Mean: baseline power level.

Variance: motion energy, higher means human movement.

Skewness: amplitude asymmetry, high for sudden events like falls.

Result: 52 × 3 = 156 features.

PCA components

Neighbouring subcarriers see almost identical reflections.

PCA projects the 52 subcarriers onto k orthogonal axes (k = 5).

Keeps high-variance motion signatures and drops uncorrelated noise.

Less redundancy, less compute.

Classifiers

SVM: finds optimal hyperplanes separating presence and room classes.

Random Forest: ensemble of decision trees voting on the class.

Input: flattened 20,800 values, or the combined statistics + PCA vector.

Role: fast benchmark before heavy networks.

Stage 3b: deep learning architectures

2D CNN

Treats the 400 × 52 matrix as a grayscale image.

2D filters slide over time and subcarriers to detect edges and power variations caused by human reflection.

Learns features without manual engineering.

EfficientNetV2

Fast, parameter-efficient CNN.

Fused-MBConv replaces depthwise convolutions in early layers to speed up training.

Progressive learning raises resolution and augmentation strength gradually.

EfficientNetV2-Small reached 97.89% on the benchmark dataset.

CNN-LSTM (target)

CNN layers extract subcarrier features at each timestep.

LSTM layers learn how the movement evolves over time.

Separates a fast fall from steady walking or an empty room.

Model implementation (PyTorch)

CNN classifier

Input: (batch, 1, 52, 400).

Conv2d 1 → 32, kernel 3 × 5, BatchNorm, ReLU, MaxPool 2 × 2.

Conv2d 32 → 64, kernel 3 × 5, BatchNorm, ReLU, MaxPool 2 × 2.

Flatten 64 × 13 × 100, FC 128, Dropout 0.5, FC with 6 outputs.

CNN-LSTM

Squeeze to (batch, 52, 400).

Conv1d 52 → 64, kernel 3, BatchNorm, ReLU, MaxPool 2 gives 64 × 200 features.

Features flattened into a 2-layer LSTM, hidden size 128, dropout 0.3.

Last LSTM output goes to a Linear layer with 6 classes.

Stage 4: evaluation metrics

Accuracy and precision

Accuracy = correct predictions / total samples.

Precision = TP / (TP + FP): how many presence alerts were real, minimizing false alarms.

Recall and F1

Recall = TP / (TP + FN): catches true presence without misses.

F1 = 2 × P × R / (P + R): harmonic mean, useful when empty-room samples outnumber zone samples.

Confusion matrix and latency

6 × 6 matrix: 1 empty state + 5 room zones, showing exact confusions such as Room 4 predicted as Room 5.

Latency = t_preprocess + t_inference in ms on edge hardware such as a laptop.

Labels and ground truth

What a label is

The ground-truth target attached to every 4-second 400 × 52 sample.

Label 0: empty room, static channel.

Labels 1 to 5: person present in Zone / Room 1 to 5.

How labels are assigned

Scripted sessions with automatic timestamp loggers, no camera needed.

Empty building for 5 minutes: all windows get Label 0.

Team member in Zone 1 for 4 minutes: all windows get Label 1.

Repeat for every zone.

Training, validation and test (8:1:1)

Training 80%

Example: 320 of 400 samples.

The model guesses a label, compares it with the true one and computes the loss.

Backpropagation finds the gradients; Adam updates the weights with momentum and RMSprop.

Validation 10%

40 samples, scored after every epoch with no weight updates.

An epoch is one full pass over the training set.

Training 99% but validation 60% means overfitting. Use it to pick the best epoch and tune learning rate and dropout.

Test 10%

40 completely unseen samples.

The best saved model runs on them once.

Final accuracy, precision, recall and confusion matrix.

Golden rule: split by session, never by window

The trap

Adjacent 4 s windows from one recording share nearly identical multipath background.

A random split puts one in train and the next in test.

The model memorizes the room, giving a fake 99% that fails in the live demo.

The fix

Record at least two separate sessions per class.

Session 1 (morning) goes to train and validation.

Session 2 (afternoon, or another day or room layout) goes entirely to test.

Test accuracy then reflects real-world performance.

Libraries and training setup

Python libraries

pip install numpy pandas scipy scikit-learn matplotlib seaborn torch torchvision

numpy, pandas: arrays, CSV loading.

scipy: Hampel, Butterworth bandpass, STFT.

scikit-learn: SVM, RF, metrics, splits.

torch, torchvision: CNN, CNN-LSTM, EfficientNet.

matplotlib, seaborn: spectrograms, loss curves, confusion matrices.

Training loop

Tensors shaped (N, 1, 52, 400); DataLoader with batch size 16 and shuffle.

CrossEntropyLoss with Adam, learning rate 1e-4, 100 epochs.

Loss printed every 10 epochs.

Final evaluation with classification_report and confusion_matrix on the test loader.

Code: Hampel filter

Stage 1: outlier suppression

```
import numpy as np
import pandas as pd

def hampel_filter_1d(input_series, window_size=5, n_sigmas=3):
    new_series = input_series.copy()
    k = 1.4826 # Gaussian scale factor
    rolling_median = new_series.rolling(window=window_size, center=True).median()
    diff = (new_series - rolling_median).abs()
    rolling_mad = k * diff.rolling(window=window_size, center=True).median()
    outliers = diff > n_sigmas * rolling_mad
    new_series[outliers] = rolling_median[outliers]
    return new_series

def clean_csi_matrix(csi_data, window_size=5): # (N_packets, 52)
    cleaned_csi = np.zeros_like(csi_data)
    for col in range(csi_data.shape[1]):
        series = pd.Series(csi_data[:, col])
        cleaned = hampel_filter_1d(series, window_size=window_size)
        cleaned_csi[:, col] = cleaned.bfill().ffill().to_numpy()
    return cleaned_csi
```

Code: phase sanitization and windowing

Phase sanitization

```
# 1. Unwrap
unwrapped = np.unwrap(raw_phase)

# 2. Remove linear slope and offset
idx = np.arange(1, 53)
slope, intercept = np.polyfit(
    idx, unwrapped, 1)
sanitized = unwrapped - (
    slope * idx + intercept)

# 3. Two-antenna alternative
sanitized_csi = csi_ant1 * np.conj(csi_ant2)
```

Windowing + min-max

```
from sklearn.preprocessing import MinMaxScaler

def create_spectrogram_samples(csi_stream,
        labels_stream, window_size=400,
        step_size=200):
    X, y = [], []
    n = csi_stream.shape[0]
    for s in range(0, n - window_size + 1,
                   step_size):
        e = s + window_size
        w = csi_stream[s:e, :] # (400, 52)
        w = MinMaxScaler().fit_transform(w)
        lab = pd.Series(
            labels_stream[s:e]).mode()[0]
        X.append(w); y.append(lab)
    return np.array(X), np.array(y)
```

Code: classical ML features

Statistics + PCA

```
from scipy.stats import skew
from sklearn.decomposition import PCA

def extract_statistical_features(m):
    mean_f = np.mean(m, axis=0)
    var_f = np.var(m, axis=0)
    skew_f = skew(m, axis=0)
    return np.hstack(
        [mean_f, var_f, skew_f]) # 156

def extract_pca_features(m, n_components=5):
    pca = PCA(n_components=n_components)
    z = pca.fit_transform(m)
    return z.flatten()
```

Dataset features + classifier

```
def create_dataset_features(
        X_raw, n_pca_components=5):
    feats = []
    for m in X_raw: # (400, 52)
        stats = extract_statistical_features(m)
        pca_f = extract_pca_features(
            m, n_pca_components)
        feats.append(np.hstack([stats, pca_f]))
    return np.array(feats)

# Train a classical model
X_tr = create_dataset_features(X_train_raw)
clf = RandomForestClassifier().fit(
    X_tr, y_train)
```

Code: 2D CNN classifier

Layers

```
import torch.nn as nn
import torch.nn.functional as F

class CSI_CNN_Classifier(nn.Module):
    def __init__(self, num_classes=6):
        super().__init__()
        # Input: (Batch, 1, 52, 400)
        self.conv1 = nn.Conv2d(1, 32,
            kernel_size=(3, 5), padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.pool1 = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(32, 64,
            kernel_size=(3, 5), padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.pool2 = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(64 * 13 * 100, 128)
        self.fc2 = nn.Linear(128, num_classes)
        self.dropout = nn.Dropout(0.5)
```

Forward pass

```
    def forward(self, x):
        x = self.pool1(F.relu(
            self.bn1(self.conv1(x))))
        x = self.pool2(F.relu(
            self.bn2(self.conv2(x))))
        x = x.view(x.size(0), -1) # flatten
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        return self.fc2(x)
```

Code: CNN-LSTM (spatial-temporal)

Layers

```
class CSI_CNN_LSTM(nn.Module):
    def __init__(self, num_classes=6,
                 hidden_dim=128, lstm_layers=2):
        super().__init__()
        self.feature_extractor = nn.Sequential(
            nn.Conv1d(52, 64, kernel_size=3,
                      padding=1),
            nn.BatchNorm1d(64), nn.ReLU(),
            nn.MaxPool1d(kernel_size=2))
        self.lstm = nn.LSTM(64 * 200, hidden_dim,
            num_layers=lstm_layers,
            batch_first=True, dropout=0.3)
        self.fc = nn.Linear(hidden_dim, num_classes)
```

Forward pass

```
    def forward(self, x):
        # Input: (Batch, 1, 52, 400)
        x = x.squeeze(1) # (Batch, 52, 400)
        b = x.size(0)
        f = self.feature_extractor(x)
        # (Batch, 64, 200) -> flatten
        f = f.view(b, 1, -1)
        out, (hn, cn) = self.lstm(f)
        # last LSTM timestep output
        return self.fc(out[:, -1, :])
```

Code: dataset and training loop

Dataset and DataLoader

```
from torch.utils.data import Dataset, DataLoader

class CSISpectrogramDataset(Dataset):
    def __init__(self, X_data, y_data):
        # (N, 1, 52, 400)
        self.X = torch.tensor(
            X_data, dtype=torch.float32
        ).unsqueeze(1).transpose(2, 3)
        self.y = torch.tensor(
            y_data, dtype=torch.long)
    def __len__(self):
        return len(self.y)
    def __getitem__(self, i):
        return self.X[i], self.y[i]

train_loader = DataLoader(
    CSISpectrogramDataset(X_train, y_train),
    batch_size=16, shuffle=True)
```

Training loop

```
def train_and_evaluate(model, train_loader,
        test_loader, epochs=100, lr=1e-4,
        device="cuda"):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(
        model.parameters(), lr=lr)
    for epoch in range(epochs):
        model.train(); running = 0.0
        for x, y in train_loader:
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad()
            loss = criterion(model(x), y)
            loss.backward()
            optimizer.step()
            running += loss.item() * x.size(0)
        if (epoch + 1) % 10 == 0:
            print(epoch + 1,
                running / len(train_loader.dataset))
```

Code: evaluation

Inside train_and_evaluate, after training

```
from sklearn.metrics import classification_report, confusion_matrix

model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for inputs, labels in test_loader:
        outputs = model(inputs.to(device))
        preds = torch.argmax(outputs, dim=1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.numpy())

print(classification_report(all_labels, all_preds, digits=4))
print("Confusion Matrix:\n", confusion_matrix(all_labels, all_preds))
```

Summary checklist for the presentation

Preprocessing stack

Hampel filter suppresses raw impulse noise.

Min-max scaling normalizes amplitude grids into [0, 1].

Data strategy

Scripted sessions with automatic timestamp loggers.

Session-based splitting keeps whole sessions in the test set and avoids false 99% claims.

Models

Baseline: EfficientNetV2 / 2D CNN, 97.89% on DP_NLOS.

Target: CNN-LSTM for spatial subcarrier features and temporal fall-motion sequences.